

Engaging Networks Regive

Standalone Engaging Networks component that enables a customizable One-Click Instant Donation experience on your "Thank You" pages. Designed to integrate seamlessly with the Engaging Networks donation form, this component can be added to any donation or premium page. It allows donors to repeat their gift with a single click—no need to refill the entire form.

No third-party libraries are required, and the component is fully customizable to match your branding and design preferences.

Table of Contents

- [Engaging Networks Regive](#)
 - [Table of Contents](#)
 - [Installation](#)
 - [Basic Usage](#)
 - [Configuration Options](#)
 - [Amount Options](#)
 - [Text Customization](#)
 - [Visual Styling](#)
 - [Theme Options](#)
 - [Advanced Options](#)
 - [Custom Theming](#)
 - [Available Merge Tags](#)
 - [Test Mode](#)
 - [Debug Mode](#)
 - [Confetti Customization](#)
 - [Utility CSS Classes](#)
 - [.showif-regive-enabled](#)
 - [.showif-regive-success](#)
 - [Regive Examples](#)
 - [Local Test Pages](#)
 - [Running the pages](#)
 - [The two paths through the component](#)
 - [Panel controls](#)
 - [Testing theme rules](#)
 - [What the pages fake](#)
 - [Development](#)

Installation

1. Install dependencies and build the Regive component:

```
npm install
npm run build:prod
```

2. Upload the generated `dist/regive.min.js` file to your Engaging Networks account.

3. Add the script to your page templates in Engaging Networks:

```
<script src="/path/to/regive.min.js"></script>
```

Note: It is important that the Regive script gets loaded on **both** the first page of your donation form and the Thank You page. This is because the Regive component needs to be initialized on the first page to understand the details of the donation being made and then embed an iframe on the Thank You page to display the regive component. If you are testing Regive and don't want to add it to your page template just yet, you can add it via Code Blocks on the first page and the Thank You page of your donation form.

Basic Usage

Add the `<regive>` tag to your "Thank You" page in Engaging Networks. The component will only work on Donation/Premium Gift pages. Here's a basic example:

```
<regive
  amount="5,8,10"
  heading="Make another impact today!"
  thank-you-message="Thank you for your additional gift!"
></regive>
```

Configuration Options

The Regive component can be customized using various attributes on the `<regive>` tag.

Amount Options

Option	Description	Example	Default
amount	Comma-separated list of donation amounts or percentages to display as buttons	amount="5,8,10"	"5"
button-label	Label for the donation buttons (can include the <code>{{amount}}</code> merge tag)	button-label="Donate <code>{{amount}}</code> Now"	"Add <code>{{amount}}</code> "

About Percentages

Percentages allow you to specify donation amounts as a percentage of the user's previous gift. For example, `amount="50%"` will set the donation amount to 50% of the user's last gift (A user gifts \$50, the Regive box would then ask for \$25). You can mix percentage amounts with fixed amounts ie. `amount="50%,10,35"`. Fixed tokens preserve their authored order, duplicates, and formatting. Percentage tokens resolve in place and are omitted if their calculated value duplicates a fixed amount or an earlier calculated result, so fewer buttons may render.

When no usable initial gift is available, an explicit `min-amount` provides the fallback for percentage tokens; otherwise percentage tokens are skipped, except for the test-mode preview behavior described below. If no valid amount remains, the established \$5 default applies. Initial gifts over `100000` are treated as unavailable. An additional Regive ask of exactly `100000` is allowed; fixed or calculated asks over `100000` are skipped, so they are not rendered or submitted.

Text Customization

Option	Description	Example	Default
heading	Message displayed above the donation buttons	heading="Double your impact today!"	null
thank-you-message	Message shown after successful donation	thank-you-message="Thank you for your additional support!"	"Thank You!"
source	Source code for the donation (saved in supporter.appealCode)	source="REGIVE-CHRISTMAS"	"REGIVE"

Visual Styling

Option	Description	Example	Default
<code>bg-color</code>	Background color of the regive component	<code>bg-color="#f5f5f5"</code>	<code>"#FFF"</code>
<code>txt-color</code>	Text color of the regive component	<code>txt-color="#0d1117"</code>	<code>"#333"</code>
<code>button-bg-color</code>	Background color of the buttons	<code>button-bg-color="#0077cc"</code>	<code>"#007BFF"</code>
<code>button-txt-color</code>	Text color of the buttons	<code>button-txt-color="#FEFEFE"</code>	<code>"#FFF"</code>

Theme Options

Option	Description	Example	Default
<code>theme</code>	Layout theme for the regive component	<code>theme="button-right"</code>	<code>"stacked"</code>
<code>theme-rules</code>	Define which theme is used based on gift amount	<code>theme-rules="500:large-donor,1000:major-donor"</code>	<code>null</code>

Available themes:

- `stacked` : Buttons stacked vertically
- `button-right` : Heading on left, buttons on right
- `button-left` : Buttons on left, heading on right
- `button-top` : Heading on top, buttons below

About theme rules

Theme rules are values paired between colons (:) and separated by commas.

For example, the rule `500:large-donor,1000:major-donor` means that if the gift amount is 500 or more, the `large-donor` theme will be applied, and if the gift amount is 1000 or more, the `major-donor` theme will be applied. Any gift amount below the first threshold will use the theme specified by the `theme` attribute or the default built-in theme, if no `theme` attribute is provided.

By default, there are no theme rules applied, and the component will use the theme specified by the `theme` attribute or the default built-in theme if no `theme` attribute is provided. If a rule names a theme that doesn't exist (neither a built-in theme nor a `<template>` on the page), the component falls back to the theme attribute, and finally to the default `stacked` theme, logging a warning in debug mode.

Advanced Options

Option	Description	Example	Default
<code>confetti</code>	Enable/disable confetti or set custom colors	<code>confetti="#FF0000,#00FF00,#0000FF"</code>	<code>"default"</code>
<code>test</code>	Enable test mode without actual form submission	<code>test="true"</code>	<code>false</code>
<code>digital-wallets</code>	Enable digital wallet payment methods	<code>digital-wallets="true"</code>	<code>false</code>
<code>params</code>	Additional URL parameters to pass to the iFrame	<code>params="utm_source=thank_you&utm_medium=regive&utm_campaign=spring"</code>	<code>null</code>
<code>base-page</code>	EN page ID to process the donation through	<code>base-page="12345"</code>	Same as original donation page
<code>ignore-required-fields</code>	Comma-separated list of mandatory field names to ignore when empty	<code>ignore-required-fields="supporter.firstName,supporter.phoneNumber"</code>	<code>null</code>

Option	Description	Example	Default
<code>gift-amount</code>	Insert for the amount the supporter initially gave	<code>gift-amount="{receipt_data~amount~[en1]}"</code>	<code>null</code>
<code>min-amount</code>	Minimum amount for a percentage gift to be	<code>min-amount="1"</code>	<code>1</code>
<code>max-amount</code>	Maximum amount for a percentage gift amount to be	<code>max-amount="100"</code>	<code>null</code>
<code>rounding-tiers</code>	Comma-separated list of rounding tiers for percentage gifts	<code>rounding-tiers="0:1,50:5"</code>	<code>0:1,50:5</code>

Base Page Option

By default, Regive processes the additional donation through the same page as the original donation. However, you can specify a different base page for processing the regive donation using the `base-page` attribute. This allows you to route the additional donation through a specific page that may have different settings, configurations, or tracking parameters. In this case, the `base-page` should be set to the page ID of the desired page in Engaging Networks.

Note: When using the `base-page` option, make sure that the specified page is properly set up to handle the donation (Contains all necessary fields for Engaging Networks to process the donation), and that it has the Regive script included on both pages. All instructions for "Page 1" of the donation form (such as adding the custom theme template) should be followed for the specified base page.

Min and Max Amounts

The values for min and max amounts only apply to percentage/dynamic amounts.

Gift Amount Format

The `gift-amount` attribute and the amounts (`min-amount` , `max-amount` , `rounding-tiers`) expect US-style number formatting (e.g. `gift-amount="$1,250.00"`). European-style formats (e.g. `€1.250,00`) are not currently supported - a value like that will not parse as intended.

Rounding tiers

Rounding tiers are groups of amounts and rounding rules for percentage gifts. Each tier is defined by a minimum amount and the rounding increment. For example, `rounding-tiers="0:1,50:5"` means that amounts from 0 to 49 will be rounded up, and amounts from 50 onwards will be rounded up to the next \$5 increment. The tiers are calculated based on gift amount, not calculated amounts.

Custom Theming

You can create custom themes by:

1. Creating a template element with an ID that will be used as the theme name. **Important:** This template must be added to **Page 1 of your Engaging Networks Donation Page**, as it will be used when the donor starts the regive donation process. For pages that use Captcha, the template must include an area for the captcha to be rendered. Here's an example of a custom theme template:

```

<template id="my-custom-theme">
  <style>
    .my-custom-theme {
      background-color: {{bg-color}};
      color: {{txt-color}};
      padding: 20px;
      border-radius: 8px;
    }
    .my-custom-buttons {
      display: flex;
      justify-content: space-between;
    }
    .my-custom-buttons button {
      background-color: {{button-bg-color}};
      color: {{button-txt-color}};
      border: none;
      padding: 10px 20px;
      border-radius: 5px;
      cursor: pointer;
    }
  </style>
  <div class="my-custom-theme">
    {{heading}}
    <div class="regive-captcha-container"></div>
    <div class="my-custom-buttons">{{button}}</div>
  </div>
</template>

```

2. Use the theme ID in your regive tag on the Thank You page:

```
<regive amount="8" theme="my-custom-theme"></regive>
```

Available Merge Tags

The following merge tags can be used in custom templates:

Merge Tag	Description
{{heading}}	Inserts the heading content
{{button}}	Inserts the donation buttons
{{ask-amount}}	The first (or only) amount button's value, formatted in USD (e.g. \$5 , \$5.01 , \$1,250)
{{theme}}	The current theme name
{{bg-color}}	The background color
{{txt-color}}	The text color
{{button-bg-color}}	The button background color
{{button-txt-color}}	The button text color

{{ask-amount}} is replaced everywhere in the template, including inside <style> blocks and CSS pseudo-element content values.

Additionally, the {{amount}} merge tag can be used in the button-label attribute to include the donation amount in the button text.

Test Mode

When you add Regive to your page, you can only see the Regive Component on your Thank You page if you have already submitted a **credit card donation**. That means that, if you access the Thank You page without having submitted a donation, you will not see the Regive Component. To test the Regive component without submitting a donation, you can enable test mode. This allows you to see the component and test all its features without making an actual donation. This is especially useful for testing the visual appearance and functionality of the component before going live. To enable test mode, add the `test` attribute to the `<regive>` tag:

```
<regive amount="5" test="true"></regive>
```

This will enable test mode for the Regive component.

When test mode is enabled:

- The component will display "Test Mode" on the right side
- Clicking a donation button will simulate the donation process
- The thank you message will appear after a simulated delay
- No actual donation will be processed
- The submission will reset after 8 seconds

Previewing dynamic (percentage) asks

In test mode, percentage amounts need a gift amount to calculate against. You can preview a specific donor gift with the `gift-amount` attribute — the full pipeline (guardrails, rounding tiers, dedupe, and theme rules) runs from that value:

```
<!-- Previews a $500 donor without a real donation -->
<regive amount="20%,50%" gift-amount="500" test="true"></regive>
```

If no gift amount is available (no `gift-amount` attribute and no stored donation), test mode uses a **default preview gift of \$50** so dynamic asks render realistically. To preview the fallback state instead (percentages resolved against the minimum), set `min-amount` explicitly:

```
<!-- Previews the fallback state: both buttons resolve to the $5 minimum -->
<regive amount="20%,50%" min-amount="5" test="true"></regive>
```

Changing the `gift-amount` value updates both the amount buttons and the theme selected by `theme-rules`.

Test mode is useful for:

- Verifying your configuration
- Testing the visual appearance
- Ensuring the thank you message displays correctly
- Satisfying your craving for confetti 🎉

Test different payment methods

You can also specify a payment method to test using the `test-method` attribute. This allows you to simulate how the Regive component behaves with different payment methods (e.g., card, Apple Pay, Google Pay) without needing to go through the actual payment process. Test must be enabled to use this feature. **NOTE:** Methods other than `card` are unable to simulate the test celebration process, they will submit to the form with an actual transaction.

```
<regive amount="5" test="true" test-method="stripedigitalwallet"></regive>
```

Accepted values for `test-method` include:

- `card`
- `applepay`
- `googlepay`
- `stripedigitalwallet`
- `paypaltouch`
- `daf`

Keep in mind that the browser, configured gateways, and page builder settings you are using for testing must support the specified payment method in order to see the corresponding behavior in the Regive component.

Debug Mode

To enable debug mode, add `debug` to the script source:

```
<script src="/path/to/regive.min.js?debug"></script>
```

Debug mode will:

- Show detailed console logs
- Display information about component initialization
- Show events and state changes
- Help troubleshoot integration issues

Confetti Customization

The confetti animation can be customized or disabled entirely:

```
<!-- Custom confetti colors -->
<regive confetti="#FF0000,#00FF00,#0000FF"></regive>

<!-- Disable confetti -->
<regive confetti="false"></regive>
```

The default confetti uses a bunch of colors that Fernando thinks are nice for your carnival.

Utility CSS Classes

Regive provides two utility CSS classes that allow you to conditionally show content based on the component's state:

.showif-regive-enabled

Content with this class will only be displayed when Regive is enabled on the page (when it's successfully loaded and initialized).

```
<!-- This content will only appear when Regive is enabled -->
<div class="showif-regive-enabled">
  <h1>
    Give a Little More.<br />
    Help a <span style="color: #de3831">Lot</span> More.
  </h1>
</div>
```

.showif-regive-success

Content with this class will only be displayed after a successful Regive donation has been made.

```
<!-- This content will only appear after a successful additional donation -->
<div class="showif-regive-success">
  <h2>Amazing! You've made an additional impact today!</h2>
  <p>We've sent another confirmation email to your inbox.</p>
  <p>Thank you for your generosity!</p>
</div>
```

These utility classes are useful for:

- Adding instructions or context around the Regive component
- Showing additional content after a successful donation

Regive Examples

Here's a complete example with various configuration options:

```
<!-- Basic example with custom styling and analytics parameters -->
<regive
  amount="6"
  heading="Make twice the impact!"
  thank-you-message="Thank you for your additional support!"
  button-label="Donate {{amount}} more"
  bg-color="#f8f9fa"
  txt-color="#212529"
  button-bg-color="#28a745"
  button-txt-color="#ffffff"
  theme="button-right"
  confetti="#28a745,#20c997,#6c757d"
  source="SPRING_REGIVE"
  params="utm_source=thank_you&utm_medium=regive&utm_campaign=spring2025"
>
</regive>
```

Here's a more complex example with custom theming and test mode:

```

<!-- Test mode example with custom theme --> THIS SHOULD GO ON PAGE 1 -->
<template id="kitten-theme">
  <style>
    .kitten-regive {
      background-color: {{bg-color}};
      color: {{txt-color}};
      padding: 20px;
      border-radius: 8px;
    }
    .kitten-regive img {
      max-width: 100%;
      max-height: 300px;
      border-radius: 8px;
      height: auto;
      object-fit: cover;
    }
    .kitten-regive button {
      background-color: {{button-bg-color}};
      color: {{button-txt-color}};
      border: none;
      padding: 10px 20px;
      border-radius: 5px;
      cursor: pointer;
    }
    .kitten-regive-disclaimer {
      margin-top: 15px;
    }
    .kitten-regive-disclaimer p {
      text-align: center;
      font-size: 0.8rem;
    }
  </style>
  <div class="kitten-regive">
    <div class="kitten-regive-img">
      
    </div>
    {{heading}}
    <p>
      You've already made a meaningful impact—thank you! With a small additional
      gift today, you can symbolically adopt a kitten and help us provide
      shelter, food, and care for these adorable animals.
    </p>
    <div class="regive-captcha-container"></div>
    {{button}}
    <div class="kitten-regive-disclaimer">
      <p>
        By clicking the button above, you'll quickly process a new donation to
        your payment method!
      </p>
    </div>
  </div>
</template>

<!-- THIS SHOULD GO ON THE THANK YOU PAGE -->
<regive
  amount="8"
  button-label="give {{amount}} more"

```

```
heading="Adopt a kitten today!"
thank-you-message="You have a new furry friend! 🐱"
theme="kitten-theme"
test="true"
>
</regive>
```

Here's an example of a regive block with dynamic amounts:

```
<regive
  amount="8,15,25%,50%"
  gift-amount="{receipt_data~amount~[en1]}"
  min-amount="1"
  max-amount="100"
  button-label="give {{amount}} more"
  heading="Adopt a kitten today!"
  thank-you-message="You have a new furry friend! 🐱"
  theme="kitten-theme"
  test="true"
>
</regive>
```

Local Test Pages

The repo ships a pair of static pages that stand in for an Engaging Networks donation form and its Thank You page, so you can run Regive end to end without an Engaging Networks account:

- `test-page-1.html` is the mock donation form (page 1 of 2). It carries the Regive script, the VGS token fields, the mandatory supporter fields, and the custom theme `<template>` elements.
- `test-thank-you.html` is the mock Thank You page (page 2 of 2). It carries the `<regive>` tag and a panel for editing every attribute without touching the file.

Both pages mirror the field names, class names, and `pageJson` values the component queries, so Regive takes the same code paths it takes on a real page. They are local fixtures for development, not something to upload to Engaging Networks.

Running the pages

1. Build the bundle the pages load:

```
npm run build
```

2. Serve the repo root over HTTP:

```
python3 -m http.server 8000
```

3. Open `http://localhost:8000/test-thank-you.html`

`npm run dev` works as well; open the same filenames on the Vite port (`http://localhost:5173/test-thank-you.html`). Opening the files over `file://` does not work: Regive needs `localStorage` and a same-origin iframe, both of which browsers block there.

The two paths through the component

Test mode is on by default in `test-thank-you.html`, so the banner renders on load, and clicking an amount simulates the donation, celebrates, and resets after 8 seconds. Nothing is submitted and no card is needed. This is the fast path for checking layout,

themes, amounts, and copy.

For the real submission path, turn test mode off and start on page 1. Write the card tokens, submit, and land on the Thank You page. Clicking an amount there submits the chained form, the iframe navigates to the Thank You page the way Engaging Networks would, and Regive matches its stored `regive-submitted` flag before reporting success to the parent.

Panel controls

The panel on the Thank You page edits every `<regive>` attribute, rewrites the URL, and shows the tag it produced, ready to paste into an Engaging Networks code block. It also logs the `postMessage` traffic coming out of the embed, lists the `regive-*` `localStorage` keys, and names the theme Regive settled on.

Harness settings are separate from the tag's attributes. They live in URL parameters and persist in `localStorage` under `regive-test-harness`, so the chained iframe inherits them:

Parameter	Default	What it does
<code>debug</code>	<code>1</code>	Appends <code>?debug</code> to the script source in both frames
<code>script</code>	<code>dist/regive.js</code>	Which bundle to load, so you can point at <code>dist/regive.min.js</code> to test the production file
<code>currency</code>	<code>USD</code>	Selects <code>transaction.paycurrency</code> , which sets the currency symbol on the buttons
<code>captcha</code>	<code>0</code>	Renders a mock reCAPTCHA, with a button that fires the callback Regive wraps
<code>wallets</code>	<code>0</code>	Renders a mock <code>#en_digitalWallet</code> block and a Stripe <code>paymentRequest</code> stub
<code>prefill</code>	<code>1</code>	Fills the mandatory supporter fields in the chained iframe, standing in for session prefill
<code>fail</code>	<code>0</code>	Fakes a failed Engaging Networks submission, so you can watch Regive exit

Testing theme rules

`test-page-1.html` carries a ladder of four custom themes built for exercising `theme-rules`:

Template id	Tier
<code>tier-supporter</code>	Base tier, below every threshold
<code>tier-sustainer</code>	\$100+
<code>tier-leader</code>	\$500+
<code>tier-champion</code>	\$1,000+

The "tier ladder" preset in the panel wires them up as `theme="tier-supporter"` with `theme-rules="100:tier-sustainer,500:tier-leader,1000:tier-champion"`. The row of gift amounts next to it steps a donor gift across the thresholds; its direct `$49.99` and `$50` controls are useful for checking the default `0:1,50:5` rounding-tier boundary. The readout names the template that rendered, so you can confirm the rule that won rather than inferring it from the layout. Two presets cover the awkward cases: "broken rule" points a rule at a theme that does not exist so you can watch the fallback chain, and "no rules" clears them.

Remember that test mode substitutes its \$50 preview gift when no gift amount is available, so rules still evaluate against \$50. Setting `min-amount` previews the fallback state instead, and rules are ignored there.

All five templates on page 1, the four tiers plus a minimal `test-theme`, print `{{ask-amount}}` twice: once in their copy, once from a CSS pseudo-element. The panel reads both values back, which covers the substitution surviving inside a `<style>` block. Built-in themes get no `{{ask-amount}}` substitution, and the readout says so when one is active.

What the pages fake

Three behaviors have no static equivalent, so the pages simulate them. Each one is commented where it happens:

- **Session prefill.** Engaging Networks repopulates the supporter fields on the chained page from the donor's session. The pages replay whatever page 1 submitted. Set `prefill=0` with test mode off to watch Regive refuse to render on empty required fields.
- **DOM mutations.** Regive re-reads the form whenever the DOM changes, and real Engaging Networks pages change it constantly. The pages stamp a data attribute on the form after any scripted value change, because setting an input's `value` property produces no mutation record on its own.
- **The iframe URL.** Regive normally rewrites the `/page/<id>/donate/<n>` path back to `/1` to find page 1. These files have no such path, so the panel sets `base-page` to page 1's absolute URL.

The mock wallet block is thinner than the rest. Moving the wallet UI into the banner and attaching the Stripe submit listener both run, but no gateway is involved: a placeholder iframe satisfies the "wallet UI is ready" check, and since the component hides iframes in the embedded frame, only the mock button stays visible.

Development

1. Clone the repository
2. Install dependencies

```
npm install
```

3. Start the development server

```
npm run dev
```

4. Open a test page on the Vite port, for example `http://localhost:5173/test-thank-you.html` (see [Local Test Pages](#)). There is no index page at the root
5. Open the console to see debug messages
6. Build for production

```
npm run build:prod
```

7. The minified file will be available at `dist/regive.min.js`